LAPORAN

Tugas 2 *Distributed Digit Relay* (ROS 2 Turtlesim)

Disusun untuk memenuhi tugas Mata Kuliah Robotika (T)

Dengan Dosen Pengampu Bapak Aldinata Rizky Revanda, S.Kom., M.Kom.



Acquirel Kriswanto	5025241035
Via Hana Nurma Putri	5025241048
Reza Afzaal Faizullah T.	5025241051
Adriel Mahira Dharma	5025241097
Nabila Shafa Rahayu	5025241150
Afsal Murtaza	5025241190

INSTITUT TEKNOLOGI SEPULUH NOPEMBER

FAKULTAS TEKNOLOGI ELEKTRO DAN INFORMATIKA CERDAS

DEPARTEMEN TEKNIK INFORMATIKA

2026/2027

1. Pendahuluan

1.1 Latar Belakang

ROS 2 memakai arsitektur terdistribusi: program besar dipecah menjadi node-node kecil yang berkomunikasi lewat topic, service, dan action. Tugas ini melatih pemahaman tersebut lewat skenario "estafet angka" (*Distributed Digit Relay*): enam node independen, satu node per anggota, bergantian menggambar angka di satu simulator turtlesim, tanpa ada koordinator pusat dan tanpa sleep atau delay hardcoded untuk mengatur urutan.

1.2 Tujuan

1. Membuat enam node ROS 2 (Python) yang masing-masing dikontrol satu anggota.
2. Menyinkronkan giliran menggambar antar node hanya melalui topic dan custom message.
3. Menggambar angka 1 sampai 12 (dua siklus, N = 6) dengan warna pena dan slot posisi berbeda untuk tiap anggota.
4. Membersihkan kanvas di antara dua siklus memakai service /clear, lalu mengakhiri relay dengan rapi setelah angka 12 (semua node masuk state idle).
5. Mengelola kode secara kolaboratif di GitHub (commit per akun, review silang, testing integrasi).

1.3 Ruang Lingkup

Relay berjalan pada satu kura-kura (turtle1) di turtlesim_node. Siklus 1 menggambar angka 1–6 oleh Anggota 1–6. Saat Anggota 1 mendapat angka 7, kanvas dibersihkan, lalu siklus 2 menggambar angka 7–12. Setelah angka 12, relay berhenti dan semua node masuk state idle.

2. Landasan Teori

Konsep	Peran dalam tugas ini
Node	Tiap anggota menjalankan satu proses node (member1 … member6).
Topic (publish/subscribe)	Topic /turn membawa giliran dari satu node ke node berikutnya.
Custom message	Turn.msg membawa angka yang harus digambar dan ID anggota yang mendapat giliran.
Service	/turtle1/set_pen, /turtle1/teleport_absolute, dan /clear dipakai untuk mengontrol turtlesim.
Callback group dan executor	ReentrantCallbackGroup dan MultiThreadedExecutor mencegah deadlock saat service dipanggil dari dalam callback.

Konsep	Peran dalam tugas ini
Launch file	relay.launch.py menjalankan turtlesim dan enam node sekaligus, dengan parameter dari YAML.
Parameter	Slot, warna pena, dan batas angka dibaca dari config/members.yaml.

3. Rancangan Sistem

3.1 Struktur Package

Repo di-clone sebagai folder src di workspace ~/ros2_ws dan berisi dua package:

Package	Tipe build	Isi
digit_relay_interfaces	ament_cmake	Custom message Turn.msg. Harus ament_cmake karena custom msg tidak bisa dibuat di package Python.
digit_relay	ament_python	Semua node anggota, modul helper, launch file, dan config.

```
~/ros2_ws/src/
|-- README.md
|-- digit_relay_interfaces/
|   |-- CMakeLists.txt
|   |-- package.xml
|   `-- msg/Turn.msg
`-- digit_relay/
    |-- package.xml, setup.py, setup.cfg
    |-- resource/digit_relay
    |-- launch/relay.launch.py
    |-- config/members.yaml
    `-- digit_relay/
        |-- __init__.py
        |-- turtle_utils.py
        |-- digits.py
        |-- number_drawer.py
        |-- relay_node.py
        |-- termination.py
        `-- member1.py ... member6.py
```

3.2 Antarmuka Pesan

```
# Turn.msg
int32 current_number   # angka yang harus digambar oleh active_member
int32 active_member    # ID anggota yang sedang mendapat giliran (1-6)
```

Nilai active_member = 0 dipakai sebagai sinyal selesai (DONE_SIGNAL).

3.3 Arsitektur Node

Semua node adalah turunan satu base class, RelayNode (relay_node.py). Perbedaan antar anggota hanya pada parameter di YAML dan *hook* yang di-override:

Hook	Fungsi	Dipakai oleh
is_starter()	Menandai node yang memulai siklus pertama	member1
before_draw(number)	Dipanggil sebelum menggambar	member1 (transisi /clear saat angka 7)
after_draw(number)	Dipanggil setelah menggambar; False berarti tidak oper giliran	member6 (terminasi setelah angka 12)

3.4 Alur Giliran

1. Setiap node subscribe ke /turn. Node hanya bertindak jika active_member sama dengan ID-nya; selain itu diam (idle).
2. Node yang mendapat giliran menggambar current_number di slotnya, lalu mempublish Turn(current_number + 1, anggota berikutnya).
3. Anggota berikutnya dihitung dengan member_id % total_members + 1, sehingga Anggota 6 mengoper kembali ke Anggota 1.
4. Setelah angka 12 digambar oleh Anggota 6, node tersebut tidak mengoper giliran, melainkan mengirim Turn dengan active_member = 0.

Angka	1	2	3	4	5	6	7	8	9	10	11	12
Anggota	1	2	3	4	5	6	1	2	3	4	5	6
Siklus	1	1	1	1	1	1	2	2	2	2	2	2

3.5 Layout Kanvas dan Warna Pena

Layout 2 baris × 3 kolom. Setiap digit berukuran 1.0 × 2.0 dengan jarak antardigit 0.5, sehingga angka dua digit selebar 2.5. Titik acuan adalah pojok kiri bawah slot. Layout dicek otomatis oleh digits.check_layout() (tidak keluar kanvas aman 0.5–10.5 dan tidak overlap); hasil pengecekan pada konfigurasi default: **valid**.

Anggota	Slot (x, y)	Warna pena	RGB
1	(1.0, 7.0)	Merah	255, 0, 0
2	(4.5, 7.0)	Oranye	255, 165, 0
3	(8.0, 7.0)	Kuning	255, 255, 0

Anggota	Slot (x, y)	Warna pena	RGB
4	(1.0, 2.0)	Hijau	0, 200, 0
5	(4.5, 2.0)	Biru	0, 120, 255
6	(8.0, 2.0)	Ungu	200, 0, 255

4. Implementasi

4.1 Pembagian Modul

Anggota	Modul inti	Node sendiri
1	Setup kedua package (package.xml, setup.py, setup.cfg, CMakeLists.txt) dan Turn.msg	member1.py: slot 1, memulai siklus, transisi /clear saat angka 7
2	turtle_utils.py (set pen, teleport bertahap dengan feedback pose, menunggu service siap) dan transisi /clear	member2.py: slot 2
3	digits.py (segmen digit 0–9 gaya 7-segmen, pengecekan layout)	member3.py: slot 3
4	relay_node.py (base class estafet: idle, cegah dobel-trigger, oper giliran, kickoff)	member4.py: slot 4
5	number_drawer.py (draw_number(): angka 2 digit, urutan garis, spasi, skala) dan relay.launch.py	member5.py: slot 5
6	termination.py (terminasi setelah angka 12), members.yaml, perangkaian README dan video demo	member6.py: slot 6, memanggil terminasi

4.2 turtle_utils.py (Anggota 2)

Kelas TurtleClient membungkus tiga service turtlesim dan subscription /turtle1/pose.

- wait_for_services() menunggu set_pen, teleport_absolute, dan /clear siap, dengan polling wait_for_service, bukan delay.
- teleport(x, y) memanggil service lalu menunggu sampai /turtle1/pose melaporkan posisi tujuan (toleransi 0.01, batas aman 5 detik).
- line_to(x, y, step) memecah garis menjadi langkah kecil (default 0.05) agar proses menggambar terlihat.
- pen_up(), pen_down(), set_pen(), dan clear() untuk kontrol pena dan kanvas.
- transition_to_next_cycle() mengangkat pena lalu memanggil /clear.

Alasan teknis feedback pose. Service teleport_absolute hanya mengantrikan perpindahan (diproses di siklus update turtlesim berikutnya), sedangkan set_pen langsung berlaku. Tanpa menunggu pose, pena bisa turun sebelum kura-kura berpindah dan meninggalkan coretan liar. Dengan feedback pose, kecepatan menggambar ditentukan siklus update turtlesim, bukan sleep.

4.3 digits.py (Anggota 3)

Setiap digit didefinisikan lewat tujuh segmen a–g seperti display 7-segmen:

```
 aaa
f   b
 ggg
e   c
 ddd
```

Digit	Segmen	Digit	Segmen
0	abcdef	5	afgcd
1	bc	6	afgedc
2	abged	7	abc
3	abgcd	8	abcdefg
4	fgbc	9	abcdfg

Fungsi pendukung: segment_points() (koordinat absolut segmen), number_width(), slot_bbox(), dan check_layout().

4.4 relay_node.py (Anggota 4)

Base class RelayNode membaca parameter dari YAML, membuat TurtleClient, menunggu service turtlesim siap, lalu baru membuat publisher dan subscriber /turn. Mekanisme penting:

- **Idle:** callback on_turn langsung return bila active_member bukan ID node.
- **Cegah dobel-trigger:** pesan dengan number <= last_drawn atau melebihi max_number diabaikan, ditambah threading.Lock non-blocking agar satu node tidak menggambar dua kali bersamaan.
- **Oper giliran:** pass_turn() mempublish Turn(number + 1, next_member()).
- **Kickoff tanpa delay:** node starter memakai timer polling dan baru mempublish Turn(1, member_id) ketika get_subscription_count() pada /turn sudah sama dengan total_members. Karena subscriber dibuat setelah service turtlesim siap, jumlah subscriber sekaligus menjadi penanda bahwa semua node benar-benar siap.
- **Executor:** MultiThreadedExecutor(4 thread) dan ReentrantCallbackGroup, supaya pemanggilan service dari dalam callback tidak deadlock.
- **Idle setelah selesai:** setelah menerima active_member = 0, node menandai finished = True dan mengabaikan semua giliran berikutnya. Fungsi run() memakai executor.spin(), sehingga node tetap hidup dalam state idle tanpa proses shutdown di akhir relay.

4.5 number_drawer.py (Anggota 5)

- plan_strokes() menyusun segmen digit menjadi garis yang sebisa mungkin bersambung (ujung segmen sebelumnya menjadi awal segmen berikutnya) sehingga pena jarang diangkat. Fungsi ini murni dan bisa dites tanpa ROS.
- digit_origins() menghitung posisi x tiap digit dengan spasi DIGIT_GAP.
- draw_digit() mengangkat pena dan teleport ke awal garis bila tidak bersambung, lalu menurunkan pena dan menggambar bertahap.
- draw_number(turtle, n, x, y, scale, step) menggambar angka 1 atau 2 digit lalu mengangkat pena.

Fungsi plan_strokes() mengurutkan segmen agar sebanyak mungkin tersambung, sehingga jumlah tarikan pena (pena diturunkan) lebih sedikit daripada jumlah segmen. Contohnya, angka 8 terdiri dari 7 segmen tetapi hanya membutuhkan 2 tarikan pena (skala 1.0):

Digit	Jumlah segmen	Tarikan pena
0	6	1
1	2	1
2	5	1
3	5	2
4	4	2
5	5	2
6	6	2
7	3	1
8	7	2
9	6	2

Selain itu, Anggota 5 membuat relay.launch.py yang menjalankan turtlesim_node dan enam node anggota dengan parameters=[members.yaml].

4.6 Setup, Config, dan Terminasi

Anggota 1 (setup). setup.py mendaftarkan launch/ dan config/ pada data_files serta enam entry point member1–member6 pada console_scripts. package.xml milik digit_relay memuat dependensi rclpy, turtlesim, std_srvs, digit_relay_interfaces, launch, dan launch_ros. Package interface memakai rosidl_generate_interfaces.

Anggota 1 (node). member1 menandai diri sebagai starter dan, pada before_draw, memanggil transition_to_next_cycle() bila number == total_members + 1 (angka 7).

Anggota 6 (config). config/members.yaml memuat member_id, total_members (6), max_number (12), slot_x, slot_y, pen_r/g/b, pen_width (3), scale (1.0), dan draw_step (0.05). Kunci teratas harus sama dengan name node pada launch file, dan nilai float wajib memakai titik desimal.

Anggota 6 (terminasi). termination.py:

1. Setelah angka 12 digambar, finish_relay() mempublish Turn(active_member = 0) dan Anggota 6 tidak mengoper giliran lagi.
2. Semua node, termasuk Anggota 6, menerima sinyal itu, menandai finished = True, lalu masuk state idle dan tidak memproses giliran lagi.
3. Node tetap hidup sehingga jendela turtlesim dan log tetap dapat diamati; seluruh sistem dihentikan dengan Ctrl+C.

4.7 Node member2 – member5

Node Anggota 2, 3, 4, dan 5 hanya mewarisi RelayNode tanpa override hook; perilaku khususnya (slot dan warna) berasal dari parameter YAML. Anggota 1 dan 6 meng-override hook seperti dijelaskan di atas.

5. Cara Menjalankan

```
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install
source install/setup.bash
ros2 launch digit_relay relay.launch.py
```

Jendela turtlesim dibiarkan terbuka setelah relay selesai agar hasil bisa dilihat; tutup dengan Ctrl+C. Untuk menutup semuanya otomatis:

```
ros2 launch digit_relay relay.launch.py shutdown_on_finish:=true
```

Memantau estafet giliran dari terminal lain:

```
ros2 topic echo /turn
```

Pengecekan layout tanpa ROS:

```
python3 -m digit_relay.digits   # mencetak: Layout valid: True
```

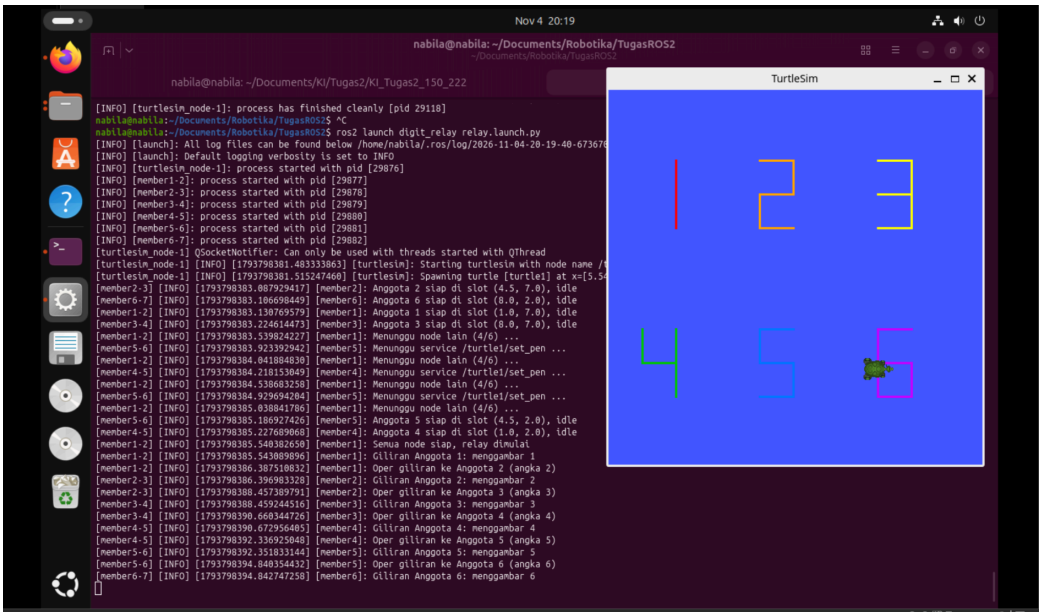
6. Hasil dan Pengujian

6.1 Skenario Uji

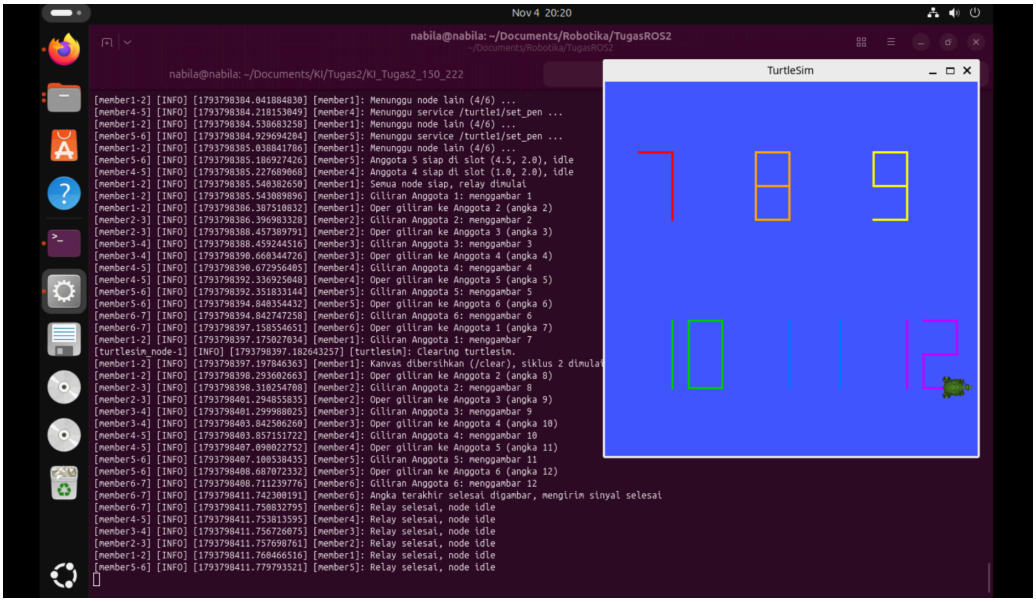
No	Skenario	Hasil yang diharapkan	Hasil
1	Start serentak lewat launch file	Tidak ada node yang menggambar sebelum keenam node siap; log "Semua node siap, relay dimulai"	Berhasil. Log "Menunggu node lain (4/6)" muncul saat dua node belum siap, lalu "Semua node siap, relay dimulai" (Gambar 1)

2	Siklus 1 (angka 1–6)	Enam angka muncul berurutan di slot dan warna yang benar, tanpa coretan liar	Berhasil. Angka 1–6 tergambar di slot dan warna masing-masing tanpa coretan liar (Gambar 1)
3	Transisi siklus	Saat angka 7, pena diangkat dan /clear membersihkan kanvas	Berhasil. Log "Clearing turtlesim" dan "Kanvas dibersihkan (/clear), siklus 2 dimulai" muncul saat angka 7 (Gambar 2)
4	Siklus 2 (angka 7–12)	Angka 10, 11, 12 (dua digit) tampil utuh, tidak overlap dan tidak keluar kanvas	Berhasil. Angka 10, 11, dan 12 tampil utuh di baris bawah (Gambar 2)
5	Terminasi	Setelah angka 12, semua node berhenti; Anggota 6 berhenti terakhir	Berhasil. Keenam node mencatat "Relay selesai, node idle"; pesan terakhir di /turn adalah active_member: 0 (Gambar 2 dan 3)

6.2 Dokumentasi Hasil



Gambar 1 : Siklus 1 (Angka 1-6)



Gambar 2 : Siklus 2 (Angka 7-12)



Gambar 3 : Output ros2 topic echo /turn

6.3 Analisis

- **Sinkronisasi tanpa delay.** Urutan giliran murni ditentukan pesan /turn; kesiapan node ditentukan polling service dan jumlah subscriber, sehingga relay tidak bergantung pada tebakan waktu.
- **Kualitas gambar.** Feedback pose pada setiap teleport mencegah pena menggambar sebelum kura-kura sampai. Konsekuensinya menggambar lebih lambat bila draw_step kecil; nilai ini bisa diubah di YAML untuk menyeimbangkan kehalusan dan kecepatan.
- **Ketahanan.** Base class tunggal menjaga perilaku seragam di enam node, dan filter last_drawn dirancang untuk mencegah dobel-trigger.

- **Keterbatasan.** Seluruh node berbagi satu turtle1, sehingga satu-satunya sinkronisasi antar node adalah token giliran di /turn. Bila satu node mati di tengah relay, estafet berhenti karena tidak ada mekanisme timeout giliran.
- **Kesiapan berbasis jumlah subscriber.** Relay dimulai saat jumlah subscriber /turn mencapai enam. Subscriber lain yang aktif sebelum launch, misalnya ros2 topic echo, ikut terhitung dan bisa membuat relay dimulai sebelum semua node siap. Karena itu echo dijalankan setelah relay dimulai, sehingga pesan untuk angka 1–3 tidak tertangkap pada Gambar 3.
- **Terminasi idle.** Versi awal menghentikan setiap node secara penuh (shutdown executor) setelah angka 12, tetapi rclpy memunculkan pesan "The following exception was never retrieved: cannot use Destroyable because destruction was requested" karena masih ada callback yang antre saat executor dimatikan. Terminasi kemudian diubah menjadi state idle, yang juga diizinkan oleh ketentuan tugas, sehingga pesan tersebut tidak muncul lagi.

7. Kolaborasi Tim

7.1 Alur Kerja

1. Bersama-sama menyepakati antarmuka: field `Turn.msg`, nama topic, signature fungsi helper, `draw_number(turtle, n, x, y, scale, step)`, dan format YAML.
2. Anggota 1 push setup lebih dulu; setelah itu semua anggota pull.
3. Modul inti dan node dikerjakan paralel.
4. Testing integrasi bersama; bug dilaporkan ke pemilik file dan diperbaiki oleh pemilik itu.

7.2 Kontribusi Anggota

Anggota	Kontribusi
1	Setup package, `Turn.msg`, member1 (starter dan /clear)
2	`turtle_utils.py`, member2
3	`digits.py`, member3
4	`relay_node.py`, member4
5	`number_drawer.py`, `relay.launch.py`, member5
6	`termination.py`, `members.yaml`, README, video demo, member6

8. Kesimpulan

Sistem *Distributed Digit Relay* berhasil dijalankan sebagai enam node ROS 2 independen yang berbagi satu turtlesim, dengan dua siklus penggambaran (angka 1–12) seperti terlihat pada Gambar 1 dan 2. Giliran diatur sepenuhnya lewat topic /turn dengan custom message Turn (Gambar 3), kesiapan sistem dideteksi lewat polling service dan jumlah subscriber (tanpa delay hardcoded), dan hasil gambar yang bersih dijamin oleh teleport bertahap dengan feedback pose.

Siklus 1 dan 2 dipisahkan oleh service /clear, dan relay berakhir lewat sinyal active_member = 0 setelah angka 12, lalu semua node masuk state idle. Pembagian modul per anggota dan rotasi review membuat seluruh anggota terlibat pada kode sendiri sekaligus kode anggota lain.